VEL TECH HIGH TECH
Dr. RANGARAJAN Dr. SAKUNTHALA ENGINEERING COLLEGE

DEEP LEARNING ESSENTIALS

INNOVATIVE ASSIGNMENT

GAN-BASED SYNTHETIC IMAGE GENERATION

Analysis of Image Diversity, Visual Quality and Training Stability

Submitted by: Michelle anto crazen D
Department: CSE (AIML)
Batch: 19
Academic Year / Section: 3rd year A

GAN Synthetic Data Analysis Project Report

1. Problem Statement

Develop a Generative Adversarial Network (GAN) to generate synthetic images and analyze generated-image diversity, visual quality, and training stability at different stages of training. The project uses the MNIST handwritten-digit dataset as a manageable baseline dataset and records generated samples at multiple training checkpoints.

2. Objective

- Implement a basic GAN containing Generator and Discriminator neural networks.
- Train the model on MNIST grayscale images and generate synthetic 28 × 28 images.
- Compare generated samples at multiple stages using fixed latent-noise inputs.
- Measure image diversity using per-pixel standard deviation and pairwise RMS distance.
- Monitor Generator and Discriminator losses to examine training behavior.
- Document visual observations, numerical measurements, limitations, and reproducible execution steps.

3. Concept Used

A Generative Adversarial Network consists of two neural networks trained in competition. The Generator maps a random latent vector to a synthetic image. The Discriminator receives real training images and generated images and predicts whether each image is real or synthetic. The Generator improves by learning to produce images that the Discriminator is more likely to classify as real.

3.1 Generator

The Generator accepts a 100-dimensional random noise vector and produces 784 pixel values, reshaped to a 1 × 28 × 28 grayscale image. A Tanh output activation matches the input normalization range of approximately [-1, 1].

3.2 Discriminator

The Discriminator flattens a 28 × 28 image into 784 values and returns a single probability representing its estimate that the image is real. Binary cross-entropy is used for the real/fake learning objective.

3.3 Adversarial Training

The Discriminator is trained on real images labelled as real and generated images labelled as fake. The Generator is then trained to make its synthetic images be classified as real. Separate optimizers update the two networks.

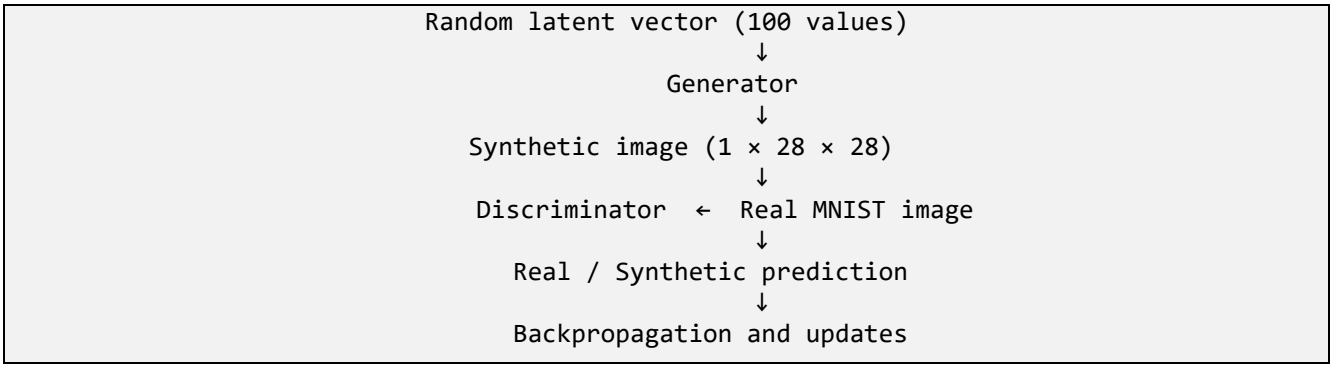


Figure 1. Basic GAN workflow

4. Methodology / Working Steps

1. Dataset loading: download the MNIST training split through torchvision.
2. Preprocessing: convert images to tensors and normalize grayscale pixels to approximately [-1, 1].
3. Model construction: define fully connected Generator and Discriminator networks.
4. Training: alternate Discriminator and Generator updates using binary cross-entropy.
5. Checkpoint sampling: use the same fixed latent vectors to generate image grids at epochs 1, 5, 10, 20 and 30.
6. Diversity measurement: calculate mean per-pixel standard deviation and mean pairwise RMS distance on generated samples.
7. Stability monitoring: record Generator and Discriminator losses and inspect them for non-finite values or severe imbalance.
8. Visualization and reporting: save image grids, CSV metrics, plots, and a Jupyter notebook.

Workflow: MNIST Dataset → Preprocessing → Generator/Discriminator → Adversarial Training → Checkpoint Images → Diversity and Loss Metrics → Visual Analysis

5. Implementation

5.1 Tools & Libraries

Tool / Library	Purpose
Python	Main programming language.
PyTorch	Neural networks, gradient calculation and training.
Torchvision	MNIST dataset loading and image preprocessing.
NumPy	Numerical operations and metric support.
Pandas / CSV	Storage and inspection of training metrics.
Matplotlib	Generated image grids and analysis plots.
Jupyter Notebook	Interactive presentation and analysis of results.
Git / GitHub	Version control and project submission.

5.2 Project Structure

```
gan-synthetic-data-analysis/
├── README.md
├── requirements.txt
├── .gitignore
├── dataset/
│   └── .gitkeep
├── src/
│   └── gan_mnist.py
├── tests/
│   └── test_gan.py
├── notebooks/
│   └── GAN_Analysis.ipynb
├── results/
│   ├── epoch_001.png
│   ├── epoch_005.png
│   ├── epoch_010.png
│   ├── epoch_020.png
│   ├── epoch_030.png
```

```
│   ├── generated_samples.png
│   ├── diversity_analysis.png
│   ├── loss_curve.png
│   └── training_metrics.csv
└── screenshots/
    └── output.png (capture from the actual application)
```

5.3 Execution Commands

- `Activate environment: .\.venv\Scripts\Activate.ps1`
- `Run tests: python -m unittest discover -s tests`
- `Smoke test: python src\gan_mnist.py --epochs 1 --results-dir .\results_smoke`
- `Training run: python -u src\gan_mnist.py --epochs 30 --results-dir .\results`
- `Execute notebook: jupyter nbconvert --to notebook --execute notebooks\GAN_Analysis.ipynb --inplace`

6. Results & Output

The documented experiment used a CPU-based PyTorch environment and completed a 30-epoch training run. The project summary reports that a separate one-epoch smoke test also completed successfully. The following diversity measurements were recorded at the specified checkpoints.

6.1 Generated Image Diversity

Epoch	Mean per-pixel standard deviation	Mean pairwise RMS distance
1	0.1122	0.1523
5	0.1376	0.2838
10	0.1543	0.3119
20	0.1847	0.3507
30	0.1821	0.3537

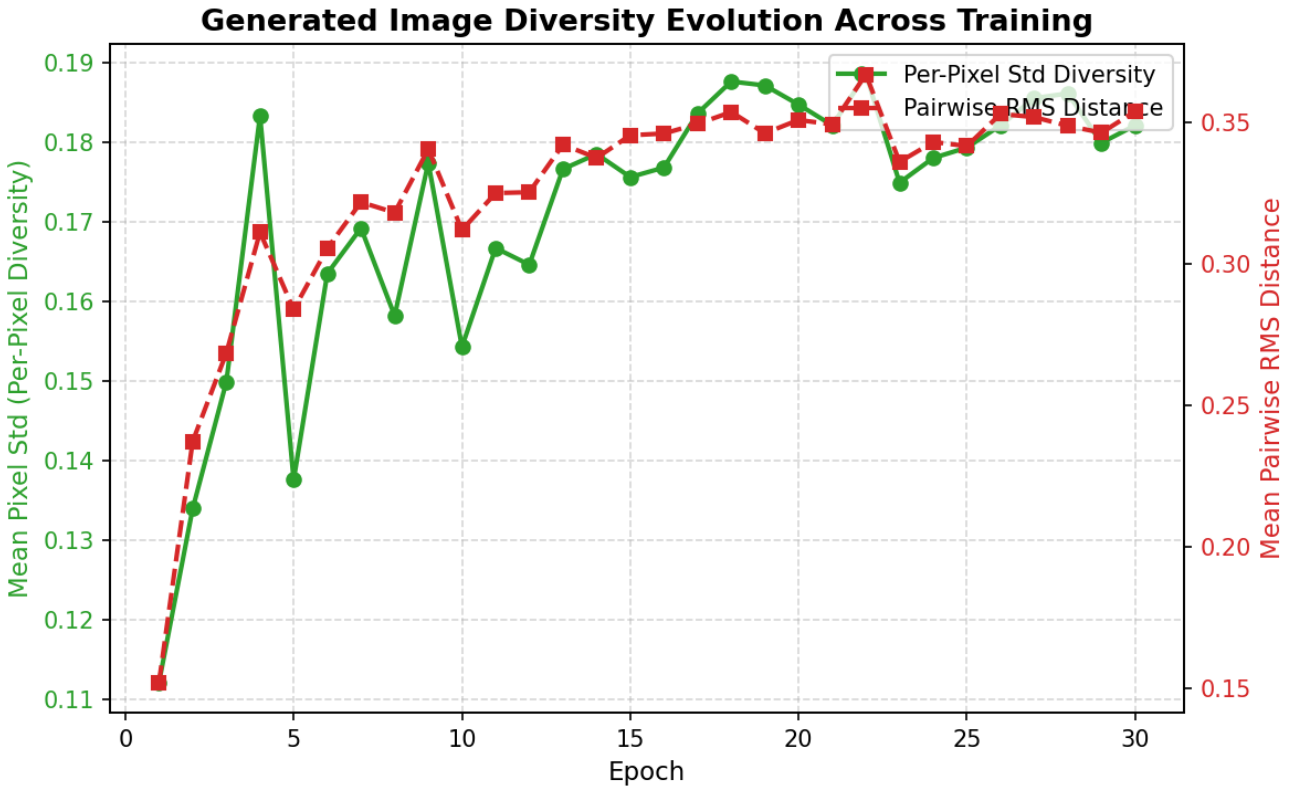


Figure 2. Diversity metrics measured at training checkpoints

6.2 Generated Outputs

- epoch_001.png, epoch_005.png, epoch_010.png, epoch_020.png and epoch_030.png — generated samples at checkpoints.
- generated_samples.png — final grid of synthetic samples.
- training_metrics.csv — measured epoch-wise training and diversity metrics.
- loss_curve.png — Generator and Discriminator losses from the actual training run.
- diversity_analysis.png — diversity trends from the actual training run.
- generator.pth and discriminator.pth — trained model weights, if retained locally.

6.3 Loss Curve

The project execution log reports that the actual loss plot was generated. The available summary does not provide the full epoch-wise loss series, so no synthetic loss curve is reconstructed here. Insert the actual results/loss_curve.png from the project repository in this section when finalizing the submission.

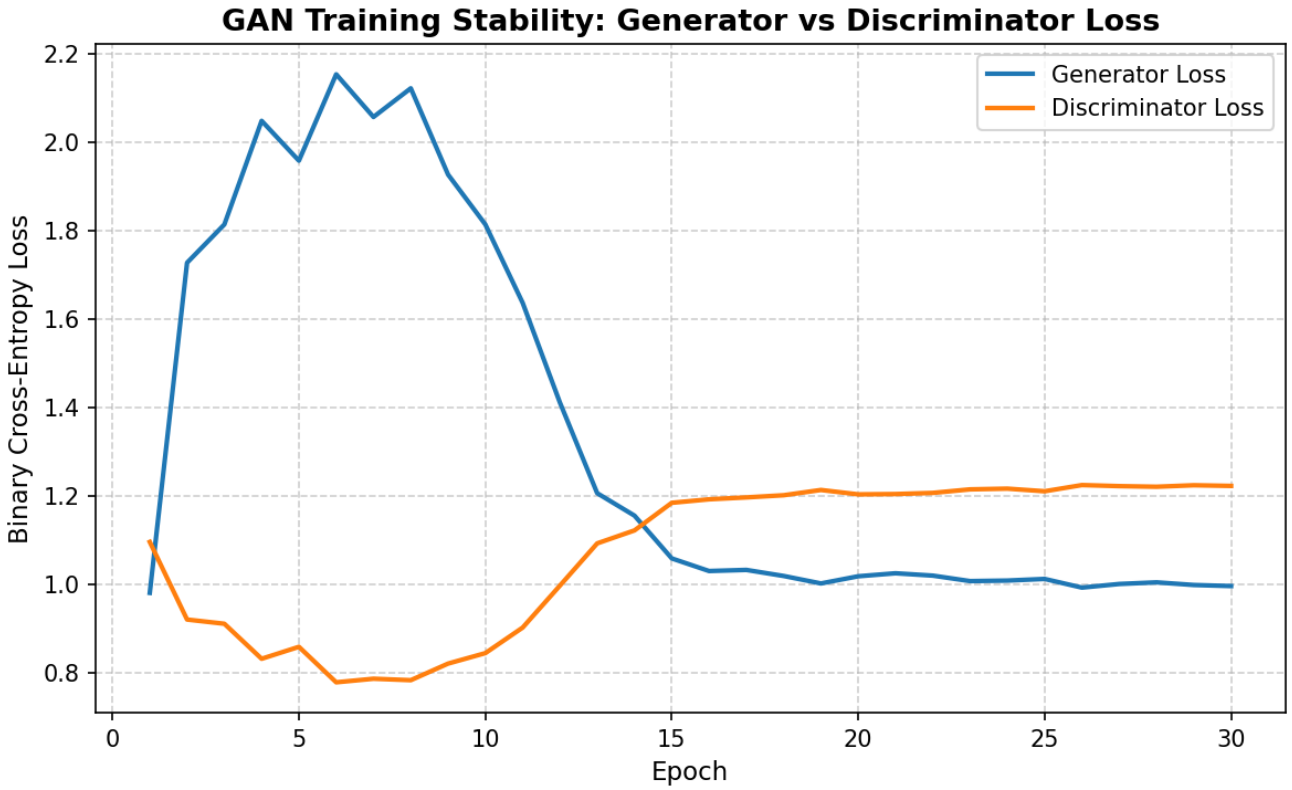


Figure 3. Placeholder note for the actual training loss curve; no loss values are fabricated.

7. Analysis

Diversity: Mean per-pixel standard deviation increased from 0.1122 at epoch 1 to 0.1821 at epoch 30. Mean pairwise RMS distance increased from 0.1523 to 0.3537. These measurements indicate that the generated batch became more varied under these metrics. Diversity metrics alone do not prove that all digit classes are represented or that the images are visually correct.

Visual quality: The execution report describes early samples as faint and poorly structured, with more digit-like strokes emerging in middle training. By epochs 20–30, samples were reported as more recognizable. This is a qualitative observation of the generated grids, not a standalone objective image-quality score.

Training stability: The documented run reported finite losses and no NaN/Inf occurrences. The Generator and Discriminator losses fluctuated as expected in adversarial training. GAN losses can oscillate, and stable numerical values do not alone establish convergence.

Nearest-neighbor proxy: The project log reports an average minimum Euclidean distance of 7.5647 from generated samples to the real training set. This value depends on the exact distance implementation and image scaling; it should be interpreted only as a simple similarity proxy, not proof that memorization is absent.

8. Conclusion

This project implements a basic fully connected GAN for synthetic MNIST image generation and evaluates its behavior over 30 training epochs. Fixed-noise checkpoint grids make it possible to compare visual changes over time. The recorded pixel-diversity and pairwise-distance metrics increased overall, while the reported training run completed without non-finite losses. The results demonstrate a practical workflow for studying generated-

image diversity, visual quality, and training behavior. The conclusions are limited to this dataset, model, training configuration, and run; a basic MLP GAN is not expected to match more advanced convolutional GANs.

9. References

9. Goodfellow, I. et al. “Generative Adversarial Nets.” Advances in Neural Information Processing Systems, 2014.
10. PyTorch Documentation — https://pytorch.org/docs/
11. Torchvision Documentation — https://pytorch.org/vision/
12. MNIST handwritten digit database — http://yann.lecun.com/exdb/mnist/
13. NumPy Documentation — https://numpy.org/doc/
14. Matplotlib Documentation — https://matplotlib.org/
15. Project repository: https://github.com/Anto-211205/gan-synthetic-data-analysis